# Day 21 : String and Date

## 1. Introduction: What is a String?

A string is a primitive data type in JavaScript used to represent a sequence of characters. Anything you can type—letters, numbers, symbols, punctuation—can be part of a string. It is the primary way we work with textual data.

**Key Characteristics:**

- **It's a primitive:** This means strings are **immutable**.
- **It's indexed:** Each character in a string has a numerical position (index), starting from zero.
- **It's an object-like primitive:** Although it's a primitive, it has methods and properties we can use, like `.length`. JavaScript temporarily wraps it in a String object when you try to access these.

---

## 2. Creating Strings

There are three ways to create a string literal in JavaScript.

1. **Single Quotes (`'...'`):**

```
let singleQuoted = 'Hello, world!';
```

2. **Double Quotes (`"..."`):** Functionally identical to single quotes. The main reason to choose one over the other is for convenience when a string itself contains quotes.

```
let doubleQuoted = "He said, 'Hello!'"; // Easy to include single quotes
let singleQuotedWithDouble = 'She replied, "Hi!"'; // Easy to include doubl
e quotes
```

3. **Template Literals (`` `...` `` - ES6):** The most powerful and modern way. They use backticks.
   We will cover the special features of template literals later.

```
let templateLiteral = `This is a template literal.`;
```

---

## 3. Core Properties and Concepts

### A. The `.length` Property

Every string has a `.length` property that tells you how many characters it contains.

```
let greeting = "Hello";
console.log(greeting.length); // Outputs: 5

let emptyString = "";
console.log(emptyString.length); // Outputs: 0
```

### B. Accessing Individual Characters (Zero-Based Indexing)

You can access a character at a specific position using square bracket notation `[]`. The first character is at index `0`.

```
let message = "JavaScript";
// J a v a S c r i p t
// 0 1 2 3 4 5 6 7 8 9

console.log(message[0]); // "J"
console.log(message[4]); // "S"

// To get the last character, a common pattern is used:
console.log(message[message.length - 1]); // "t"
```

### C. The Golden Rule: Strings are Immutable

This is the most critical concept. **You cannot change a string in place.** Any method that appears to modify a string will always **return a brand new string**, leaving the original untouched.

```
let name = "alex";

// Let's try to change the first character.
name[0] = "A"; // This will FAIL silently. It does nothing.
console.log(name); // Outputs: "alex" (The original string is unchanged)

// Let's use a method that "changes" the string.
let upperName = name.toUpperCase();
console.log(upperName); // Outputs: "ALEX" (This is a NEW string)
console.log(name);      // Outputs: "alex" (The original is still unchanged)
```

---

## 4. Common and Essential String Methods

These methods are your primary tools for working with strings. Remember, they all return **new strings**.

### A. Changing Case

- `.toUpperCase()` **:** Returns a new string with all characters in uppercase.
- `.toLowerCase()` **:** Returns a new string with all characters in lowercase.

```
let whisper = "please be quiet";
let shout = whisper.toUpperCase(); // "PLEASE BE QUIET"
```

### B. Finding Substrings

- `.indexOf(substring)` **:** Returns the index of the **first occurrence** of a substring. If the substring is not found, it returns **1**.
- `.lastIndexOf(substring)` **:** Returns the index of the **last occurrence** of a substring. Returns **1** if not found.
- `.includes(substring)` **(ES6):** Returns a boolean (`true` or `false`) indicating if the string contains the substring. This is often more readable than `indexOf`.

```
let sentence = "The quick brown fox jumps over the lazy fox.";
```

```
console.log(sentence.indexOf("fox"));     // 16 (the first one)
console.log(sentence.lastIndexOf("fox")); // 40 (the last one)
console.log(sentence.indexOf("cat"));     // -1 (not found)

console.log(sentence.includes("jumps"));  // true
console.log(sentence.includes("cat"));    // false
```

### C. Extracting Substrings

- `.slice(startIndex, endIndex)`: Extracts a section of a string and returns it as a new string.
  - `startIndex`: The index where the extraction begins (inclusive).
  - `endIndex`: The index where the extraction ends (exclusive - it does not include this character).
  - You can use negative indices, which count from the end of the string.

```
let text = "JavaScript";

console.log(text.slice(0, 4));  // "Java" (from index 0 up to, but not includin
g, 4)
console.log(text.slice(4));     // "Script" (from index 4 to the end)
console.log(text.slice(-6));    // "Script" (the last 6 characters)
```

- `.substring(startIndex, endIndex)`: Similar to `.slice()`, but it doesn't accept negative indices.
- `.substr(startIndex, length)`: **(Deprecated - Avoid)**. It works with a start index and a length, which is different from the others. Use `.slice()` instead.

### D. Replacing Substrings

- `.replace(searchValue, newValue)`: Finds a `searchValue` and replaces it with `newValue`.
  - **Gotcha:** By default, it only replaces the **first occurrence**.
- `.replaceAll(searchValue, newValue)` **(ES2021):** Replaces **all occurrences**. This is the modern, easy way to do a global replace.

```
let greeting = "hello world, hello there";

// Replaces only the first "hello"
let newGreeting = greeting.replace("hello", "hi");
console.log(newGreeting); // "hi world, hello there"

// Replaces all "hello"s
let allNewGreeting = greeting.replaceAll("hello", "hi");
console.log(allNewGreeting); // "hi world, hi there"
```

- **Old way to replace all:** Use a regular expression with the global flag (`/g`).
  `greeting.replace(/hello/g, "hi");`

### E. Cleaning Up Whitespace

- **`.trim()`:** Removes whitespace from both the beginning and end of a string.
- **`.trimStart()` / `.trimEnd()`:** Removes whitespace from only the start or end.

```
let userInput = "   my-username@example.com   ";
let cleanedInput = userInput.trim(); // "my-username@example.com"
```

### F. Splitting a String into an Array

- **`.split(separator)`:** Splits a string into an array of substrings, using the `separator` to decide where to split. This is incredibly useful.

```
let csvData = "item1,item2,item3";
let items = csvData.split(","); // ["item1", "item2", "item3"]

let words = "The quick brown fox";
let word_array = words.split(" "); // ["The", "quick", "brown", "fox"]

let letters = "abc";
let letter_array = letters.split(""); // ["a", "b", "c"]
```

---

## 5. Template Literals (ES6) - The Modern Way

Template literals, created with backticks (`` ` ``), are a massive improvement for working with strings.

**A. Variable Interpolation ( `${...}` ):**
This allows you to embed expressions and variables directly into a string. It's far more readable than using the `+` operator for concatenation.

- **The Old Way (Concatenation):**

```
let name = "Alice";
let age = 30;
let message = "Hello, my name is " + name + " and I am " + age + " years old.";
```

- **The New Way (Template Literals):**

```
let name = "Alice";
let age = 30;
let message = `Hello, my name is ${name} and I am ${age} years old.`;
// You can even put expressions inside:
let futureMessage = `Next year, I will be ${age + 1}.`;
```

**B. Multi-line Strings:**
Template literals respect newlines inside the string.

- **The Old Way:**

```
let htmlOld = '<div>\\n' +
              '  <p>Hello</p>\\n' +
              '</div>';
```

- **The New Way:**

```
let htmlNew = `
  <div>
    <p>Hello</p>
```

```
  </div>
`;
```

# Date

---

## In-Depth Lecture Notes: The JavaScript `Date` Object

## 1. The Core Concept: A Single Moment in Time

Before we look at any code, we must understand the "first thought" principle of JavaScript dates.

**A `Date` object is not a calendar; it's a single, massive number.** This number represents the total milliseconds that have passed since a universal starting point: the **Unix Epoch**, which is midnight on **January 1st, 1970, UTC** (`01-jan-1970`).

- Every date you create is just a wrapper around this timestamp.
- This makes it easy to compare dates and calculate durations.

---

## 2. Creating `Date` Objects

You create a `Date` object using the `new Date()` constructor.

**A. Create a Date for the Current Moment**
If you provide no arguments, you get the current date and time from the user's device.

```
// Creates a Date object for right now.
const now = new Date();
console.log(now); // e.g., Mon Oct 28 2024 03:30:00 GMT+0530 (India Stand
ard Time)
```

**B. Create a Date from a Timestamp**
You can create a date from that big "milliseconds" number we talked about. This is how servers and databases often communicate time.

```
// Date.now() is a quick way to get the CURRENT timestamp as a number.
const currentTimestamp = Date.now(); // e.g., 1759262295036
console.log(currentTimestamp);

// Now, let's create a Date object from that number.
const da = new Date(1759262295036);
console.log(da); // This will show the date and time corresponding to that time stamp.
```

**C. Create a Specific Date (The Recommended Way)**
This is the most reliable way to create a specific date. You provide the components as numbers in this order: `year, month, day, hours, minutes, seconds, ms`.

```
// new Date(year, month, day, hours, minutes, seconds, ms)
const myDate = new Date(2025, 8, 4, 6, 20, 11, 125);
console.log(myDate.toString()); // Thu Sep 04 2025 06:20:11 GMT+0530 (India Standard Time)
```

**🚨 CRITICAL GOTCHA: Months are Zero-Indexed! 🚨**
This is the most common source of bugs with JavaScript dates.

- `0` = January
- `1` = February
- ...
- `8` = **September** (as in the example above)
- `11` = December

---

## 3. Getting Information from a `Date` Object (Getters)

Once you have a `Date` object, you can extract its individual parts. These methods return values based on the user's local timezone.

```
const now = new Date(); // Let's pretend it's Sep 4, 2025, a Thursday

console.log(now.getFullYear()); // 2025 (The full 4-digit year)
console.log(now.getMonth());    // 8 (Remember, 0-indexed, so 8 is Septembe
r)
console.log(now.getDate());     // 4 (The day of the month, 1-31)
console.log(now.getDay());      // 4 (The day of the week: 0=Sunday, 1=Monda
y, ..., 4=Thursday)
console.log(now.getHours());    // e.g., 6 (The hour, 0-23)
console.log(now.getMinutes());  // e.g., 20 (The minute, 0-59)
```

## 4. Changing a Date: Setters and Mutability

`Date` objects are **mutable**, meaning you can change their value in place using "setter" methods.

```
const da = new Date(1759262295036); // Creates a specific date
console.log("Before:", da);

// Let's change the month to May (index 4)
da.setMonth(4);

console.log("After:", da); // The ORIGINAL 'da' object has been changed.
```

This is different from primitives like strings or numbers. Calling a setter method **mutates** the original object.

## 5. Date Auto-Correction (Rollover)

The `Date` object is smart about invalid dates. If you give it a value that's out of range, it will "roll over" to the next logical date.

Your example `new Date(2025, 1, 30)` is a perfect demonstration. You are asking for **February 30th, 2025**.

```
// 2025 is not a leap year, so February has 28 days.
// You are asking for Feb 28 + 2 days.
const a = new Date(2025, 1, 30);

// The date object auto-corrects to March 2nd, 2025.
console.log(a); // Outputs: Sun Mar 02 2025 ...
```

This feature can be useful for date math (like adding 30 days to a date), but it can also hide bugs if you aren't expecting it.

---

## 6. Formatting Dates for Display

A raw `Date` object isn't very user-friendly. You must format it into a string for display.

```
const now = new Date();

// .toString() is the default, verbose format in the local timezone.
console.log(now.toString());
// e.g., Mon Oct 28 2024 04:15:00 GMT+0530 (India Standard Time)

// .toDateString() gives you just the date part.
console.log(now.toDateString());
// e.g., Mon Oct 28 2024

// .toISOString() gives the universal, unambiguous UTC time. The 'Z' means U
TC.
// This is what you send to servers.
console.log(now.toISOString());
// e.g., 2024-10-27T22:45:00.000Z

// .toLocaleString() formats the date and time in a way that is natural
// for the user's region and language. BEST for user display.
console.log(now.toLocaleString());
```

```
// e.g., in India: 28/10/2024, 4:15:00 am
// e.g., in the US: 10/27/2024, 4:15:00 PM
```

### Summary of Key "Gotchas"

1. **Months are 0-indexed.** This is the #1 source of bugs.
2. **Dates are Mutable.** Setter methods change the original object.
3. **String Parsing is Unreliable.** Never use `new Date("some-date-string")`.
4. **Timezones are Complex.** Always be aware of whether you are working with local time or UTC time. Send UTC timestamps to servers.

```
/ console.log(now.toString());
// console.log(now.toDateString());
// console.log(now.toISOString());
// console.log(now.toLocaleString());
```

The JavaScript `Date` object is **notorious** for being one of the most criticized parts of JavaScript. Here's why:

## Major Problems:

### 1. **Months start at 0, but days start at 1** 🤦

```
new Date(2025, 0, 1);  // January 1, 2025 (month is 0-indexed)
new Date(2025, 11, 25); // December 25, 2025 (11 = December!)
// This is confusing and error-prone!
```

### 2. **Mutable (can be changed accidentally)**

```
const date = new Date(2025, 9, 1);
date.setMonth(10); // Oops! Date is modified
console.log(date); // Changed to November!
// This causes bugs in apps
```

### 3. **Weird quirks and bugs**

```
new Date(2025, 1, 30); // Feb 30? Becomes March 2!
new Date('2025-02-30'); // Invalid Date

// Inconsistent behavior
new Date(2025, 0); // Jan 1, 2025 00:00:00
new Date(2025);    // Jan 1, 1970 plus 2025 milliseconds! 😱
```

## The Solution:

Because `Date` is so bad, developers use libraries:

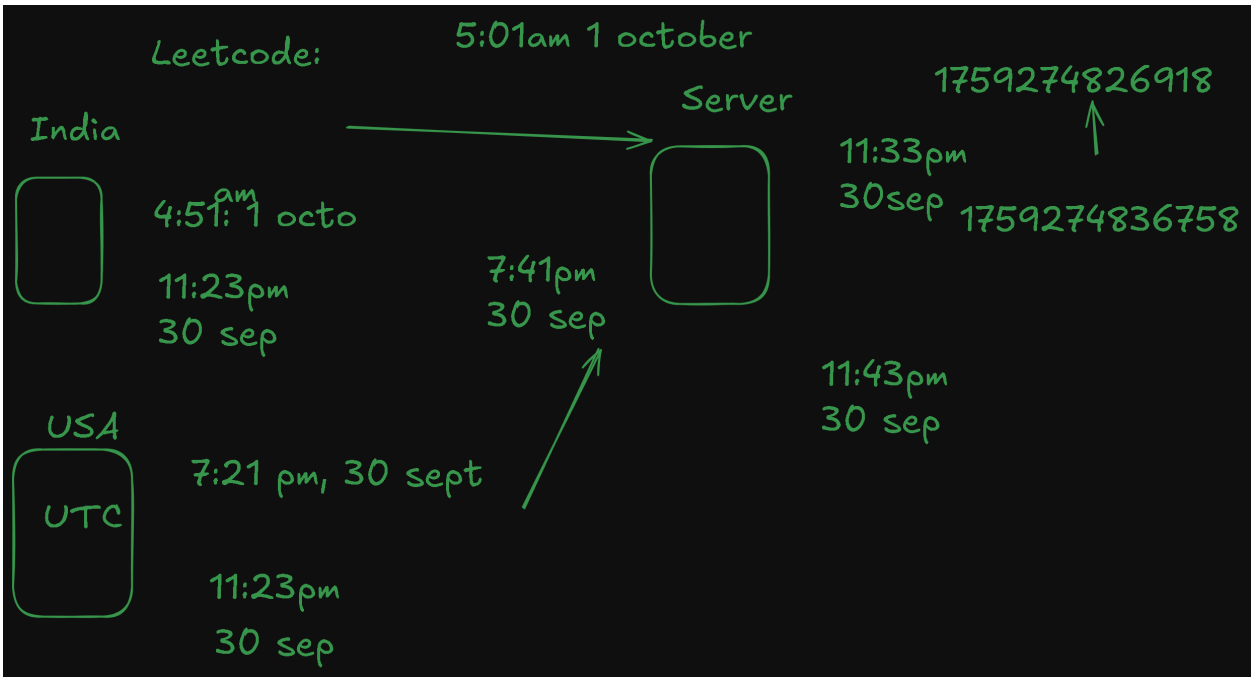
- **Day.js** (lightweight, 2KB)
- **date-fns** (functional, tree-shakeable)
- **Luxon** (modern, powerful)

## The Future: **Temporal API** ⏰

JavaScript is getting a **new date/time API called Temporal** that fixes all these issues:

```
// Much better!
const date = Temporal.PlainDate.from('2025-10-01');
date.add({ days: 3 }); // Returns new date, immutable!
date.toString(); // '2025-10-01' (clean!)
```

Temporal is currently in Stage 3 and should be available in browsers/Node.js soon!


Leetcode:
5:01am 1 october
Server
1759274826918
India
11:33pm
30sep
1759274836758
4:51 am 1 octo
11:23pm
30 sep
7:41pm
30 sep
11:43pm
30 sep
USA
7:21 pm, 30 sept
UTC
11:23pm
30 sep